

Decroly SANTANDER

UD3

API REST CON SPRING BOOT

MP_0613

Desarrollo Web en Entorno Servidor

3.4 Consumir y documentar APIs

Qué vas a aprender en este tema

Hasta ahora tu tienda ha sido siempre el **servidor**: otros le hacen peticiones y ella responde. En este tema cambiarás de papel. Primero, tu aplicación será **cliente** de servicios web de otras empresas: leerá un catálogo de productos de una API pública y consultará tipos de cambio para mostrar precios en otras monedas. Aprenderás también a reaccionar cuando esos servicios fallan, que es lo que marca la diferencia en una aplicación real. Después, **documentarás** tu propia API con el estándar OpenAPI, para que cualquier programador pueda usarla sin leer tu código.

1. Tu aplicación como cliente de otra API

Un backend también consume servicios

Ya viste que un servicio web ofrece datos a otras aplicaciones. Muy a menudo, esa "otra aplicación" es **otro backend**. Una tienda online real no hace todo por sí misma: delega en servicios especializados.

Necesidad de la tienda	Servicio web externo	Qué le pide la tienda
Cobrar	Pasarela de pago	Autorizar un cargo en una tarjeta
Enviar pedidos	Empresa de transporte	Crear un envío y consultar dónde está
Precios en otras monedas	Servicio de tipos de cambio	El tipo de cambio actual del euro
Facturar	Agencia tributaria	Registrar la factura emitida
Catálogo de un proveedor	API del proveedor	Sus productos y precios

En todos estos casos, tu aplicación actúa como **cliente**: construye una petición HTTP, la envía, espera la respuesta y la interpreta. Es exactamente lo que hacían Postman o el navegador con tu API, pero ahora desde código Java.

Tecnologías y protocolos que intervienen

Elemento	Qué papel tiene al consumir un servicio
HTTP / HTTPS	El protocolo de transporte. Casi todas las APIs públicas exigen HTTPS: la comunicación va cifrada con TLS.
URL base y rutas	Cada API tiene una dirección base (`https://dummyjson.com`) y unas rutas documentadas (`/products`).

Elemento	Qué papel tiene al consumir un servicio
Formato de datos	Normalmente JSON. Hay que convertirlo a objetos Java, igual que hace Spring con `@RequestBody`.
Autenticación	Muchas APIs exigen identificarse, normalmente con una **clave de API** en una cabecera (`X-API-Key: …`) o con un *token*. Las que usaremos son públicas.
Límites de uso	Las APIs públicas limitan cuántas peticiones puedes hacer por minuto (*rate limit*). Si te pasas, responden `429 Too Many Requests`.
Documentación	Sin ella no sabrías qué rutas existen ni qué devuelven. Por eso documentarás tu API en el apartado 3.

Los clientes HTTP de Spring

Spring ofrece tres herramientas para hacer peticiones HTTP desde Java:

Cliente	Características	Uso en este curso
`RestTemplate`	El clásico, desde 2009. Métodos como `getForObject` o `exchange`. Muy extendido en código existente.	Debes reconocerlo.
`RestClient`	El moderno. Estilo **fluido**: la petición se construye encadenando métodos. Es el recomendado para código nuevo.	**El que usaremos.**
`WebClient`	Cliente **reactivo**, del módulo Spring WebFlux. Para aplicaciones con muchísima concurrencia.	Solo para reconocerlo.

La misma petición con los dos primeros:

RestTemplate frente a RestClient

```
// RestTemplate (libro)
RestTemplate plantilla = new RestTemplate();
RespuestaProductosDto r1 = plantilla.getForObject(
        "https://dummyjson.com/products", RespuestaProductosDto.class);

// RestClient (curso)
RestClient cliente = RestClient.create("https://dummyjson.com");
RespuestaProductosDto r2 = cliente.get()      // metodo HTTP
        .uri("/products")                     // ruta
        .retrieve()                           // enviar y recibir
        .body(RespuestaProductosDto.class);   // convertir el JSON
```

Con `RestClient`, cada paso de la petición se lee en orden: método, ruta, envío y conversión. Si la API responde con un código 4xx o 5xx, `retrieve()` lanza una excepción, lo que estudiarás en el apartado 2.

Leer el JSON de otros: DTO a medida

Cuando consumes una API ajena, el formato del JSON lo decide **otro**. Por ejemplo, un producto de DummyJSON trae más de veinte campos, en inglés: `id`, `title`, `description`, `category`, `price`, `rating`, `stock`, `brand`, `images`… A ti solo te interesan cuatro. La solución es crear un **DTO** que refleje solo la parte del JSON que necesitas y pedir a Jackson que ignore el resto:

Un DTO para el producto de DummyJSON

```
@JsonIgnoreProperties(ignoreUnknown = true)
public record ProductoExternoDto(Long id, String title,
                                 double price, int stock) {
}
```

Los nombres de los campos del `record` deben coincidir con las claves del JSON (`title`, `price`). Después, en tu código, conviertes ese DTO en tu propia clase `Producto`. Así, si la API externa cambia su formato, solo cambia el DTO y la conversión, y el resto de tu aplicación queda protegido.

> **Por qué no usar directamente Producto**
> Si deserializaras el JSON de DummyJSON directamente en `Producto`, tu clase de negocio tendría que llamar a sus campos `title` y `price` porque así lo decidió otra empresa. El DTO hace de **frontera**: lo de fuera se adapta a lo de dentro, nunca al revés.

Práctica 1. Un catálogo desde una API pública

DummyJSON es una API pública de datos de prueba. Antes de programar, **explórala** como haría cualquier programador con una API nueva:

1. Abre en el navegador `https://dummyjson.com/products?limit=3`. Observa la estructura: un objeto con los campos `products` (el array), `total`, `skip` y `limit`.
2. Añade la petición a tu colección de Postman, en una carpeta nueva `APIs externas`.

Ahora crearás una **tercera implementación** de `ProductoService`, que cargue los productos iniciales desde DummyJSON:

1. Añade a `application.properties` la dirección de las APIs externas. Van en la configuración, no en el código, por si cambian:

application.properties

```
apis.catalogo-url=https://dummyjson.com
apis.divisas-url=https://api.frankfurter.dev/v1
apis.timeout-segundos=3
```

1. Crea en `config` un `record` para esas propiedades (se registrará solo, gracias a `@ConfigurationPropertiesScan`):

ApisProperties.java

```
package es.decroly.tienda.config;

import org.springframework.boot.context.properties.ConfigurationProperties;

@ConfigurationProperties(prefix = "apis")
public record ApisProperties(String catalogoUrl,
                             String divisasUrl,
                             int timeoutSegundos) {
}
```

1. Crea en `dto` los dos DTO que reflejan la respuesta de DummyJSON:

ProductoExternoDto.java

```
package es.decroly.tienda.dto;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;

@JsonIgnoreProperties(ignoreUnknown = true)
public record ProductoExternoDto(Long id, String title,
                                 double price, int stock) {
}
```

RespuestaProductosDto.java

```
package es.decroly.tienda.dto;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import java.util.List;

@JsonIgnoreProperties(ignoreUnknown = true)
public record RespuestaProductosDto(List<ProductoExternoDto> products,
                                    int total) {
}
```

Las anotaciones de Jackson, como `@JsonIgnoreProperties`, conservan el paquete `com.fasterxml.jackson.annotation` tanto en Jackson 2 como en Jackson 3.

1. Crea la implementación en `services`:

ProductoServiceApi.java

```
package es.decroly.tienda.services;

import es.decroly.tienda.config.ApisProperties;
import es.decroly.tienda.domain.Producto;
import es.decroly.tienda.dto.ProductoExternoDto;
import es.decroly.tienda.dto.RespuestaProductosDto;
import org.springframework.boot.autoconfigure.condition.ConditionalOnProperty;
import org.springframework.stereotype.Service;
import org.springframework.web.client.RestClient;

import java.util.ArrayList;
import java.util.List;

@Service
@ConditionalOnProperty(name = "tienda.productos.origen",
                       havingValue = "api")
public class ProductoServiceApi extends ProductoServiceBase {

    public ProductoServiceApi(ApisProperties apis) {
        super(cargar(apis.catalogoUrl()));
    }

    private static List<Producto> cargar(String urlBase) {
        RestClient cliente = RestClient.create(urlBase);

        RespuestaProductosDto respuesta = cliente.get()
                .uri("/products?limit={limite}", 20)
                .retrieve()
                .body(RespuestaProductosDto.class);

        // Convertir cada producto externo en un producto de la tienda
        List<Producto> lista = new ArrayList<>();
        for (ProductoExternoDto p : respuesta.products()) {
            lista.add(new Producto(p.id(), p.title(),
                                   p.price(), p.stock()));
        }
        return lista;
    }
}
```

Elemento	Explicación
`RestClient.create(urlBase)`	Crea un cliente cuyas rutas se añadirán a esa dirección base.
`.uri("/products?limit={limite}", 20)`	Las llaves son huecos que se rellenan con los argumentos siguientes, igual que en el logger. RestClient codifica los valores correctamente en la URL.
`.retrieve().body(…)`	Envía la petición, comprueba que la respuesta es 2xx y convierte el cuerpo JSON en el tipo indicado.

Elemento	Explicación
Bucle `for`	Convierte cada DTO externo en un producto de nuestra tienda: `title` pasa a ser `nombre` y `price` pasa a ser `precio`.
Carga en el constructor	La petición se hace **una vez, al arrancar**. A partir de ahí, la tienda trabaja con su copia en memoria.

1. Cambia a `tienda.productos.origen=api` (en `application.properties` o en el perfil que tengas activo), arranca y lista los productos desde Postman. Comprueba que tienen nombres en inglés y que son 20.
2. Crea un producto nuevo con POST: se añade a la copia en memoria, con el id 21. ¿Se ha creado también en DummyJSON? ¿Por qué?
3. `ProductoServiceMemoriaTest` sigue funcionando sin conexión. ¿Y la prueba `contextLoads` de `TiendaDecrolyApplicationTests`, que arranca la aplicación completa con la configuración de `application.properties`? Razónalo y compruébalo desconectando la red.
4. Cambia `apis.catalogo-url` por una dirección que no existe (`https://noexiste.decroly.es`) y arranca. La aplicación **no arranca**. Lee el error: ¿qué excepción de fondo aparece? Vuelve a dejar la dirección buena.

Actividad 2. Cuando la otra API falla

Qué puede ir mal

Cuando tu aplicación depende de un servicio que no controlas, tienes que asumir que **fallará** en algún momento. Una aplicación profesional se distingue por cómo reacciona:

Situación	Qué ocurre en RestClient	Qué debe responder tu API
La API no responde (caída, red lenta)	Se queda esperando… salvo que haya un **tiempo de espera**. Entonces lanza `ResourceAccessException`.	503 Service Unavailable
No hay conexión o la dirección no existe	`ResourceAccessException`	503 Service Unavailable

Situación	Qué ocurre en RestClient	Qué debe responder tu API
La API responde 4xx (petición incorrecta)	`HttpClientErrorException`	Depende: si el error lo causó un dato del cliente, 400
La API responde 5xx (error suyo)	`HttpServerErrorException`	503 Service Unavailable

Las tres excepciones heredan de `RestClientException`, así que se pueden capturar todas juntas o por separado.

Tiempos de espera

Por defecto, un cliente HTTP puede esperar **indefinidamente**. Si la API externa se cuelga, cada petición a tu tienda se quedaría también colgada, ocupando un hilo del servidor. Con suficientes peticiones, tu tienda dejaría de responder por culpa de otro. Por eso siempre se configuran dos tiempos:

Tiempo de espera	Qué limita
De conexión (*connect timeout*)	Cuánto se espera para establecer la conexión con el servidor remoto.
De lectura (*read timeout*)	Cuánto se espera a que llegue la respuesta una vez conectados.

Traducir el error remoto a tu API

Tu cliente no sabe (ni debe saber) que dependes de un servicio de divisas. Si ese servicio falla, no le devuelvas su error tal cual: tradúcelo a **tu** lenguaje. Crearás una excepción nueva, `ServicioExternoException`, que el manejador global convertirá en un **503 Service Unavailable**: "el problema es temporal y no es culpa tuya, inténtalo más tarde".

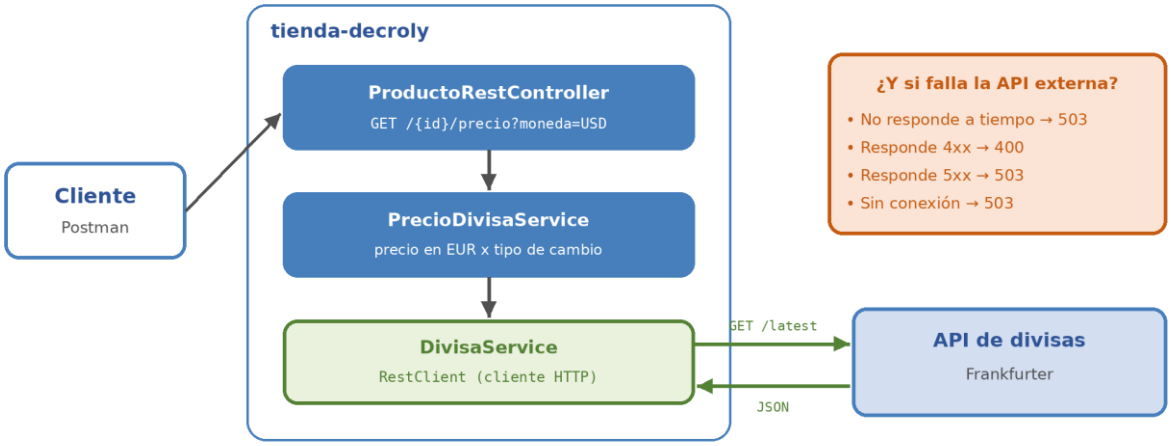


Figura 1. La tienda consulta un servicio de divisas y traduce sus posibles fallos a respuestas propias.

Práctica 2. Precios en otras monedas

Frankfurter es una API pública y gratuita que publica los tipos de cambio oficiales del Banco Central Europeo. Explórala primero:

1. Abre `https://api.frankfurter.dev/v1/latest?base=EUR&symbols=USD` en el navegador. Obtendrás algo así: `{"amount":1.0,"base":"EUR","date":"…","rates":{"USD":1.1}}`.
2. Prueba `symbols=XYZ`, una moneda que no existe, en Postman. ¿Qué código de estado devuelve?

> **Versiones de la API de Frankfurter**
>
> Frankfurter ha publicado una versión 2 de su API. La versión 1, que usamos aquí, sigue funcionando y es la que describe su documentación de forma más sencilla. Como la dirección está en `apis.divisas-url`, cambiar de versión solo afectaría a la configuración y al DTO.

1. Crea el DTO de la respuesta. Las tasas llegan como un objeto JSON cuyas claves son monedas, así que en Java se leen como un `Map`:

TipoCambioDto.java

```
package es.decroly.tienda.dto;

import com.fasterxml.jackson.annotation.JsonIgnoreProperties;
import java.util.Map;

@JsonIgnoreProperties(ignoreUnknown = true)
public record TipoCambioDto(String base, String date,
                            Map<String, Double> rates) {
}
```

1. Crea en `exceptions` la clase `ServicioExternoException` (igual que las otras dos) y añade al manejador global su tratamiento:

ManejadorGlobalErrores.java (nuevo método)

```
@ExceptionHandler(ServicioExternoException.class)
public ProblemDetail servicioExterno(ServicioExternoException ex) {
    ProblemDetail problema = ProblemDetail.forStatusAndDetail(
            HttpStatus.SERVICE_UNAVAILABLE, ex.getMessage());
    problema.setTitle("Servicio externo no disponible");
    return problema;
}
```

1. Crea el servicio que consume Frankfurter. Esta vez el `RestClient` se configura con tiempos de espera:

DivisaService.java

```
package es.decroly.tienda.services;

import es.decroly.tienda.config.ApisProperties;
import es.decroly.tienda.dto.TipoCambioDto;
import es.decroly.tienda.exceptions.PeticionIncorrectaException;
import es.decroly.tienda.exceptions.ServicioExternoException;
import lombok.extern.slf4j.Slf4j;
import org.springframework.http.client.SimpleClientHttpRequestFactory;
import org.springframework.stereotype.Service;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestClient;
import org.springframework.web.client.RestClientException;

import java.time.Duration;

@Slf4j
@Service
public class DivisaService {

    private final RestClient cliente;

    public DivisaService(ApisProperties apis) {
        Duration espera = Duration.ofSeconds(apis.timeoutSegundos());
        SimpleClientHttpRequestFactory fabrica =
                new SimpleClientHttpRequestFactory();
        fabrica.setConnectTimeout(espera);
        fabrica.setReadTimeout(espera);

        this.cliente = RestClient.builder()
                .baseUrl(apis.divisasUrl())
                .requestFactory(fabrica)
                .build();
    }

    public double tipoDeCambio(String moneda) {
        try {
            TipoCambioDto respuesta = cliente.get()
                    .uri("/latest?base=EUR&symbols={moneda}", moneda)
```

```
                    .retrieve()
                    .body(TipoCambioDto.class);
            Double tipo = respuesta.rates().get(moneda);
            if (tipo == null) {
                throw new PeticionIncorrectaException(
                        "Moneda no admitida: " + moneda);
            }
            return tipo;
        } catch (HttpClientErrorException ex) {
            // 4xx: la API no reconoce la moneda que nos han pedido
            throw new PeticionIncorrectaException(
                    "Moneda no admitida: " + moneda);
        } catch (RestClientException ex) {
            // 5xx, tiempo de espera agotado o sin conexion
            log.error("Fallo al consultar el servicio de divisas", ex);
            throw new ServicioExternoException(
                    "El servicio de divisas no esta disponible");
        }
    }
}
```

Elemento	Explicación
`RestClient.builder()`	A diferencia de `create`, permite configurar el cliente antes de construirlo.
`SimpleClientHttpRequestFactory`	La pieza que realmente abre las conexiones. En ella se fijan los tiempos de espera.
Orden de los `catch`	Primero la excepción más concreta (`HttpClientErrorException`), después la general (`RestClientException`). Al revés, el primer `catch` lo atraparía todo.
`log.error(…, ex)`	El detalle técnico del fallo queda en el log para el programador; el cliente solo recibe un mensaje claro.
El cliente se crea en el constructor	Se crea **una vez** (el servicio es un bean único) y se reutiliza en todas las peticiones.

1. Crea el DTO de la respuesta y el servicio que combina el precio con el tipo de cambio:

PrecioMonedaDto.java

```
package es.decroly.tienda.dto;

public record PrecioMonedaDto(Long idProducto, String moneda,
                              double tipoCambio, double total) {
}
```

PrecioDivisaService.java

```
package es.decroly.tienda.services;

import es.decroly.tienda.dto.PrecioDto;
import es.decroly.tienda.dto.PrecioMonedaDto;
import lombok.RequiredArgsConstructor;
import org.springframework.stereotype.Service;

@Service
@RequiredArgsConstructor
public class PrecioDivisaService {

    private final PrecioService precioService;
    private final DivisaService divisaService;

    public PrecioMonedaDto calcular(Long idProducto, String moneda) {
        PrecioDto enEuros = precioService.calcular(idProducto);
        if (moneda.equalsIgnoreCase("EUR")) {
            return new PrecioMonedaDto(idProducto, "EUR", 1.0,
                                       enEuros.total());
        }
        String codigo = moneda.toUpperCase();
        double tipo = divisaService.tipoDeCambio(codigo);
        double total = Math.round(enEuros.total() * tipo * 100) / 100.0;
        return new PrecioMonedaDto(idProducto, codigo, tipo, total);
    }
}
```

Fíjate en que no hemos modificado `PrecioService`: hemos creado un servicio nuevo que lo **reutiliza**. Así, sus pruebas del tema 3.3 siguen funcionando sin cambios.

1. Añade `PrecioDivisaService` como dependencia del controlador de productos y modifica el endpoint de precio para que acepte la moneda:

ProductoRestController.java (endpoint modificado)

```
@GetMapping("/{id}/precio")
public PrecioMonedaDto precio(@PathVariable Long id,
        @RequestParam(defaultValue = "EUR") String moneda) {
    return precioDivisaService.calcular(id, moneda);
}
```

1. Prueba en Postman `/api/productos/1/precio`, `…?moneda=USD` y `…?moneda=GBP`.
2. Prueba `…?moneda=XYZ`. Debes recibir un **400** con un `ProblemDetail` que explique que la moneda no está admitida.
3. **Simula que la API está caída:** cambia `apis.divisas-url` a `http://localhost:9999` (ahí no escucha nadie), reinicia y repite la petición con USD. Debes recibir un **503**. Mira el log: ¿qué excepción aparece?

4. **Simula que la API no responde:** cambia la dirección a `http://10.255.255.1` (una dirección que no contesta nunca). Cuenta cuánto tarda en llegar el 503. ¿Coincide con `apis.timeout-segundos`?
5. Pon `apis.timeout-segundos=30` y repite. Imagina 200 clientes haciendo esa petición a la vez: ¿qué le pasaría a tu tienda? Vuelve a dejar 3 segundos y la dirección correcta.
6. Guarda en Postman las peticiones de precio en USD, en EUR y con moneda inexistente.

3. Documentar una API

Por qué documentar

En las prácticas anteriores has consumido dos APIs sin ver ni una línea de su código. Lo has podido hacer porque estaban **documentadas**: sabías qué rutas existían, qué parámetros admitían y qué devolvían. Si otro equipo quiere usar tu API de la tienda, necesita lo mismo. Una buena documentación de una API responde, para cada endpoint:

- Qué hace, con qué método HTTP y en qué ruta.
- Qué parámetros admite (de ruta, de consulta, cuerpo) y cuáles son obligatorios.
- Qué devuelve cuando va bien, con un ejemplo.
- Qué errores puede devolver y en qué casos.

Escribir eso a mano en un documento tiene un problema: en cuanto cambias la API, la documentación queda desactualizada. La solución es **generarla a partir del código**.

La especificación OpenAPI

OpenAPI es un estándar para describir APIs REST en un fichero JSON o YAML que pueden leer tanto las personas como los programas. Antes se llamaba *especificación Swagger*, y por eso ambos nombres se usan a menudo como sinónimos.

Fragmento de una descripción OpenAPI (YAML)

```yaml
openapi: 3.1.0
info:
  title: API de la Tienda Decroly
  version: "1.0"
paths:
  /api/productos/{id}:
    get:
      tags: [Productos]
      summary: Consultar un producto
      parameters:
        - name: id
          in: path
          required: true
          schema: { type: integer }
      responses:
        "200":
          description: Producto encontrado
        "404":
          description: No existe el producto
```

A partir de ese fichero, distintas herramientas pueden:

Herramienta	Qué hace con la especificación
Swagger UI	Genera una página web interactiva con todos los endpoints, desde la que se pueden probar sin Postman.
Postman	Importa el fichero y crea automáticamente una colección con todas las peticiones.
Generadores de código	Crean automáticamente el código cliente para consumir la API en Java, JavaScript, Python…

Code-first y contract-first

Enfoque	Cómo se trabaja	Cuándo se usa
Code-first	Se programa la API y la especificación se **genera** a partir del código.	Equipos pequeños, APIs que evolucionan rápido. **El que usaremos.**
Contract-first	Primero se escribe y se acuerda la especificación, y después se programa cumpliéndola.	Varios equipos que trabajan a la vez: el del frontend puede empezar sin esperar al backend.

springdoc-openapi

springdoc-openapi es la librería que, al arrancar, inspecciona tus controladores (rutas, parámetros, tipos de retorno) y genera la especificación OpenAPI y Swagger UI. Sin escribir nada, ya obtiene mucha información. Con sus anotaciones puedes enriquecerla:

Anotación	Dónde	Para qué
`@Tag(name, description)`	Clase del controlador	Agrupa los endpoints bajo un título.
`@Operation(summary, description)`	Método	Explica qué hace el endpoint.
`@ApiResponse(responseCode, description)`	Método (se puede repetir)	Documenta cada respuesta posible, incluidos los errores.
`@Parameter(description, example)`	Parámetro	Explica un parámetro de ruta o de consulta.
`@Schema(description, example)`	Clase o atributo del dominio o de un DTO	Explica un campo y da un valor de ejemplo.
`@OpenAPIDefinition(info = @Info(…))`	Una clase de configuración	Título, versión y descripción general de la API.

Todas estas anotaciones están en el paquete `io.swagger.v3.oas.annotations` y sus subpaquetes.

Práctica 3. Documentación de la API de productos

1. Añade la dependencia a `pom.xml` y recarga Maven. Esta librería no la gestiona Spring Boot, así que **sí hay que indicar la versión**:

pom.xml

```
<dependency>
    <groupId>org.springdoc</groupId>
    <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
    <version>3.0.3</version>
</dependency>
```

ATENCIÓN · Versión de springdoc

Las versiones **3.x** de springdoc son las compatibles con Spring Boot 4; con Spring Boot 3 se usan las **2.8.x**. Usa la versión 3.x más reciente que encuentres en `https://springdoc.org` o en Maven Central.

1. Arranca la aplicación y abre `http://localhost:8080/swagger-ui.html`. **Sin haber escrito nada**, ya aparecen tus controladores. Despliega `GET /api/productos/{id}`, pulsa **Try it out**, escribe un id y pulsa **Execute**.
2. Abre `http://localhost:8080/v3/api-docs`: es la especificación en JSON que alimenta esa página. Prueba también `/v3/api-docs.yaml`.
3. Crea en `config` la información general de la API:

OpenApiConfig.java

```
package es.decroly.tienda.config;

import io.swagger.v3.oas.annotations.OpenAPIDefinition;
import io.swagger.v3.oas.annotations.info.Info;
import org.springframework.context.annotation.Configuration;

@Configuration
@OpenAPIDefinition(info = @Info(
        title = "API de la Tienda Decroly",
        version = "1.0",
        description = "Catalogo de productos y familias de la tienda"))
public class OpenApiConfig {
}
```

1. Documenta el controlador de productos. Este es el modelo para uno de los endpoints; aplícalo a todos:

ProductoRestController.java (documentación de un endpoint)

```
@Tag(name = "Productos", description = "Catalogo de productos")
@RestController
@RequestMapping("/api/productos")
@RequiredArgsConstructor
public class ProductoRestController {

    // ...

    @Operation(summary = "Consultar un producto por su id")
    @ApiResponse(responseCode = "200", description = "Producto encontrado")
    @ApiResponse(responseCode = "404",
            description = "No existe el producto",
            content = @Content(
                    schema = @Schema(implementation = ProblemDetail.class)))
    @GetMapping("/{id}")
    public Producto buscarPorId(
            @Parameter(description = "Id del producto", example = "1")
            @PathVariable Long id) {
        return productoService.buscarPorId(id);
    }
}
```

1. Documenta los campos de `Producto` para que Swagger UI muestre ejemplos realistas:

Producto.java documentado

```
@Schema(description = "Producto del catalogo de la tienda")
@Getter
@Setter
@NoArgsConstructor
@AllArgsConstructor
public class Producto {

    @Schema(description = "Identificador asignado por la tienda",
            example = "1", accessMode = Schema.AccessMode.READ_ONLY)
    private Long id;

    @Schema(description = "Nombre comercial", example = "Teclado mecanico")
    private String nombre;

    @Schema(description = "Precio sin IVA en euros", example = "49.90")
    private double precio;

    @Schema(description = "Unidades disponibles", example = "15")
    private int stock;
}
```

`accessMode = READ_ONLY` indica que el `id` aparece en las respuestas, pero no se envía al crear un producto, porque lo asigna el servidor.

1. Recarga Swagger UI y comprueba el resultado: la descripción general, el grupo "Productos", los ejemplos en **Schemas** y las respuestas de error documentadas.

2. Usa Swagger UI para crear un producto con POST (el cuerpo de ejemplo ya viene relleno) y para provocar un 404.
3. Descarga `http://localhost:8080/v3/api-docs.yaml` y guárdalo en tu proyecto como `docs/openapi.yaml`.
4. En Postman, pulsa **Import**, selecciona ese fichero y observa la colección que se genera. Compárala con la que construiste a mano en el tema 3.2.

La documentación como contrato

Ahora tu API cumple lo que pedías a DummyJSON y a Frankfurter: cualquiera puede usarla leyendo su documentación. Y como se genera del código, si cambias un endpoint, la documentación cambia con él. Lo que **no** se genera solo son las descripciones y los errores: mantener actualizadas las anotaciones es responsabilidad tuya.